

EvoVec: Evolutionary Image Vectorization with Adaptive Curve Number and Color Gradients

Egor Bazhenov¹, Ivan Jarsky¹, Valeria Efimova¹, and Sergey Muravyov¹

ITMO University, Saint-Petersburg, Russia

Abstract. Vector and raster graphics are the two main types of 2D images used in computer graphics. Raster graphics are images consisting of pixels (dots); vector images are created using mathematical objects such as lines, curves, and shapes. The main advantage of vector graphics is that they can be scaled without loss of quality, which is useful for advertising, design, frontend development, and other fields of application. At the moment, the issue of vectorization (conversion from raster to vector graphics) has not been fully resolved. There are two main approaches: deterministic algorithms and machine learning-based algorithms. Both of these types are not able to work with a color gradient and have other disadvantages, such as artifacts for deterministic algorithms, and extremely long working time and predefined curve number for machine learning-based algorithms. To solve the problems of existing solutions, we propose an evolutionary algorithm for image vectorization. Its main idea is to iteratively improve vector images using mutations and crossover. The proposed algorithm does not require any necessary parameters other than the original image and can process color gradients. The results of comparison with existing solutions show that our algorithm qualitatively and quickly vectorize images. Particularly, our approach outperforms others in terms of pixel-by-pixel MSE by 15%. The implementation is publicly available¹.

Keywords: Image vectorization · evolutionary algorithm · machine learning · vector graphics

1 Introduction

Raster and vector graphics are the primary forms of visual representation employed in computer graphics. Both types of graphics have their strengths and weaknesses and are used in various areas of design and visuals, depending on the specifics of the task.

Raster graphics or bitmaps consist of pixels (dots), each having a specific color and coordinates on the screen. The color is commonly set by three channels: red, green and blue (RGB). The bitmaps quality depends on their resolution and number of pixels, so this type of graphics is not scalable.

¹ <https://github.com/EgorBa/EvoVec-Evolutionary-Image-Vectorization>

Vector graphics are images created using mathematical objects such as lines, curves, and shapes. Each shape is called a path and is defined mathematically using a coordinate vector. Vector images can be scaled without loss of quality [2], as they do not depend on the number of pixels. Vector images are usually saved in SVG [20] format.

The process of converting a vector image to a bitmap image is called rasterization [13], and the one from bitmap to vector image is called vectorization [25]. A rasterization process has already been studied in great detail, because there are a huge number of methods for its implementation [17]. However, the task of vectorization remains relevant, since existing solutions do not yet manage this task well.

There are two main approaches for vectorization: deterministic algorithms and machine learning-based algorithms. Deterministic algorithms include existing solutions from libraries or online services [9]. Their main disadvantage is that the quality of vectorization is not high, because these algorithms do not work with a gradient colors and sometimes create artifacts on a vectorized image. In turn, machine learning-based algorithms (like LIVE [15] or DiffVG [14]) can create structured vector images, so that is their main advantage. Their disadvantages include: long working time, inability to operate with a gradient, and the requirement for knowledge of the curves number in a vectorized image.

In this paper, we propose another approach to vectorization problem, since existing solutions do not manage the task of vectorization sufficiently. We have achieved quality improvement through the use of an evolutionary algorithm. This allows to vectorize the image without additional parameters and using a gradient.

The result of this paper is an evolutionary algorithm that can vectorize a raster image using a variable curve number. Specifically, the main contributions are as follows:

1. The proposed algorithm does not require input parameters for vectorization.
2. The algorithm operates with a color gradient, which improves the vectorization quality.
3. Vectorized image does not require predefined curve number and has fewer curves than the deterministic algorithm result.
4. It works faster than existing algorithms based on machine learning.

For more information about existing solutions, see the Related Works section 2. The Methods section 3 will describe in detail the algorithm itself and the subtleties of its implementation. In the Comparisons and Experiments 4, the results of comparing the proposed algorithm operation with existing solutions and ablation study are presented. The Conclusion section 5 summarizes the paper.

2 Related Work

In this section, we summarize previous approaches and present works that are closely related to our article. The existing surveys [8,23] proposes vectorization

methods classification. According to it, the two main groups of methods are deterministic algorithms and machine learning-based algorithms. Along with these algorithms, we also discuss evolutionary approaches in this section.

2.1 Vectorization algorithms based on machine learning

There are lots of machine learning-based vectorization algorithms, however some of them are universal (designed to vectorize any images) and the others are customized for a predefined type of images. Non-universal algorithms can only be used for highly specialized tasks and therefore they are not of particular interest for the general problem of image vectorization. Among universal algorithms we selected DiffVG [14] and LIVE [15] as the most widely used [24,11] existing approaches.

DiffVG. Many machine learning-based vectorization algorithms use a process of a vector image gradual optimization so that its rasterization is similar to a bitmap image. This optimization is possible due to the method of differentiable rasterization of a vector image proposed in the DiffVG [14] paper. In this paper, the authors propose a functional operator that accepts a vector image as input and outputs its raster analog. The main purpose of this operation is to further transfer the resulting bitmap image to the loss function, to call the error back propagation algorithm and to calculate gradients for the parameters of the original vector image. Then, according to the standard practice, an optimization step can be performed to result in vector parameters updating. Thus, the vector image is changed according to the specified loss function, and if, for example, this loss function is MSE, then the vector image is optimized towards more similarity with the custom raster analog.

One of DiffVG advantages is that it does not have a rigid binding to any predefined type of vector images and it is able to process any images. However, DiffVG is not able to dynamically determine which shapes or segments should be added or removed from a vector image. The DiffVG method operates on vector images, preserving their original structure and changing only the values of the shape parameters, for example, color, point coordinates, and stroke width. This is one of the reasons why using DiffVG without additional improvements does not result in the images most closely resembling the original ones.

At the moment, the following innovations are used among the algorithms known: 1) according to a pre-selected scheme, step-by-step addition of the shapes to a vector image is applied (for example, in LIVE method), 2) DiffVG is used in neural networks for the purpose of differentiable transition from vector to raster domain, 3) DiffVG is used for final optimization of a vector image pre-generated by a separate neural network.

To the best of our knowledge, there are still no methods related to DiffVG that could remove unnecessary shapes and segments from a vector image.

Im2Vec. One of the first approaches integrating DiffVG into neural networks is the Im2Vec model [21], its main purpose is image vectorization. The model is a VAE (Variational autoencoder) transforming an input bitmap into a hidden space and then generating the parameters of the vector image corresponding to

the input bitmap from the hidden space. The network is trained using DiffVG, due to which the generated vector image is converted to a raster image and a raster loss function is applied, evaluating the similarity of the generated raster image with the original raster image. The main disadvantage of this model is that the model is able to correctly vectorize only images similar to training set. At the same time, according to our experiments, this model is not able to synthesize complex vector images consisting of a large number of shapes and segments.

LIVE. The iterative vectorization approach is used in LIVE [15] method. Before running this algorithm, the scheme for adding new shapes must be specified. This scheme determines how many new shapes will be added at each stage of optimization. Instead of applying shapes to the canvas in random places, the algorithm searches for such monochrome areas in the image that are not covered by the desired shape.

The method also introduces two new loss functions. The first penalizes the algorithm for creating self-intersecting shapes, which in practice can lead to the appearance of unnatural patterns. The second loss function seeks to estimate the coincidence of the rasterization of vector shapes with the real area of the original raster image more correctly than by MSE. MSE approach was shown to be problematic due to the following considerations. Vector images are often a composition of various shapes superimposed on each other. The LIVE algorithm suggests adding shapes to a vector image step by step, starting with the outer large shapes, rather than the inner smaller ones. However, using the mean square error (MSE) as a loss function between the current external vector shape and the real raster area containing this external shape and its internal subfigures is incorrect. Instead, it is proposed to use the UDF (User-Defined Function) loss function, which evaluates how precise a vector shape corresponds to the desired boundaries and how correct its color is, rather than the average color of this external shape and its internal ones. The proposed losses allow more effective evaluating and improving the intermediate results of the LIVE algorithm when vectorizing images.

However, the main problem with the LIVE method is its very low image generation speed. In addition, the quality of vectorized images in terms of visual similarity to the original bitmap is still not precise.

2.2 Deterministic algorithms

At the moment, a huge number of deterministic algorithms for vectorization exists [7,9,19,27]. The main shortcomings of such algorithms are vectorization quality, as results are obtained with a partial loss of initial information, and a large number of unnecessary paths used in resulting vector image. These algorithms advantages include high vectorization speed and the absence of necessary parameters tuning to improve vectorization.

In our approach, deterministic algorithms are used to get an initial population. This helps to ensure that the initial vectorization quality of our algorithm will not be worse than the quality of the deterministic ones.

2.3 Evolutionary vectorization

An evolutionary algorithm [1] is a numerical optimization method based on the principles of natural selection and evolution. To obtain more adapted individuals, evolutionary algorithms use mutations [28] and crossovers [22]. A mutation is a change in one particular individual using a special algorithm, that can improve or worsen it. Crossover is an operator which involves combining features from two parent individuals to create offspring. By exchanging features between parents, crossover promotes exploration of the search space and facilitates the creation of new potential solutions. One of the evolutionary algorithm main characteristics is the selection of the initial population [12], because all new individuals will be built on the basis of it. The important parameter of the evolutionary algorithm is the elite percentage [4] — the percentage of individuals selected from the current population for further development. The size of the population [10] has a great influence on the evolutionary algorithm quality, because the chance of getting a more adapted individual with a large population size is higher, therefore it is necessary to select the optimal value for it based on available computational power.

Vectorization of bitmaps using evolutionary algorithms was discussed in the paper [3]. The described approach has a number of significant drawbacks that do not allow it to be used for vectorization. The vectorization process has long working time, and the result is very different from the original images, because the algorithm works with discrete geometric shapes and use masks. This approach makes it possible to add different styles to vectorized images by user, but vectorized image does not have a high vectorization quality.

In our approach, we decided to use evolutionary algorithm, because image vectorization is a task of optimizing the resulting image quality. Our algorithm is more universal and makes it possible to get a vector image similar to the original.

3 Method

The main part of the proposed solution is the evolutionary algorithm, which is used to solve optimization problem. Image vectorization aims to optimize vector image to exactly repeat raster image, thus, it is an optimization task. A vectorized image is an individual to which appropriate mutation and crossover operators are applied to obtain a higher quality. For vectorization, we also create a population from various vectorization options and further improve and select only the best ones.

3.1 The initial population

Several options were considered to create the initial population. After conducting a number of experiments, it was concluded that the result of a deterministic algorithm should be copied to form an initial population of n individuals.

We also studied and tested various Python libraries for initial vectorization (Pixels2Svg ², Potrace ³, SvgTracer ⁴). SvgTracer is the best library we have found for our task, because it is the fastest and its results have the highest quality.

3.2 Mutations and crossovers

To achieve the best quality of vectorization, we use four types of mutations: needle mutation, probabilistic gradient connection of paths based on color differences, probabilistic path deletion, and probabilistic path segment deletion. As crossover, we use a random exchange of paths.

A needle mutation is a probabilistic change in the coordinates of path segments added for preventing path overgrowth and shape correction. This mutation works in such a way that the needle mutation coefficient is added or subtracted to a probabilistically selected coordinate of a path. At the same time, it is limited by the size of the image, and this mutation also has its own subtypes such as: a constant value of the coordinate change and a gradual decrease in the value by which the coordinate changes. For more complex vectorization tasks, the second type of the mutation should be used, and for the rest — the first one. In Figure 1, example A shows the problem that this mutation is designed to solve.

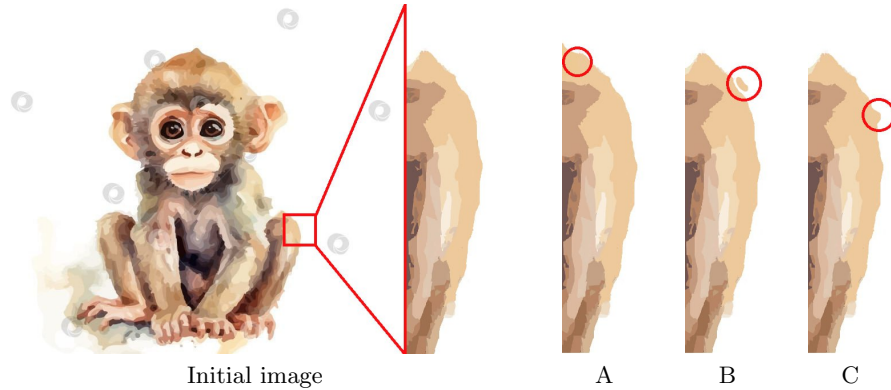


Fig. 1: Example of different vectorization problem and their solution with different mutation. A — example of incorrect initial coordinates of a path. This problem is solved by needle mutation. B — artifact of vectorization. Mutation of dropping path removes this artifact. C — extra segment in the path. Mutation of dropping segment smoothes the curve.

² <https://pypi.org/project/pixels2svg/>

³ <https://pypi.org/project/pypotrace/>

⁴ <https://pypi.org/project/svgtracer/>

While other mutation, the path is deleted with a certain degree of probability. The same approach is used for deleting the segments of the path. These mutations help to eliminate artifacts created during the initial vectorization. The mutation for deleting a path has an additional parameter responsible for the size of the deleted area, because large elements of the image should not be deleted. For example, in Figure 1, Figures B and C illustrate this issue with artifacts.

The main purpose of the fourth mutation is the gradient connection of two paths based on the difference in their colors, set by a threshold, which can be tuned [26]. This mutation is applied with a certain probability, and the connection occurs using a gradient [6] from the color of the first path to the color of the second. Figure 2 shows how the algorithm works.

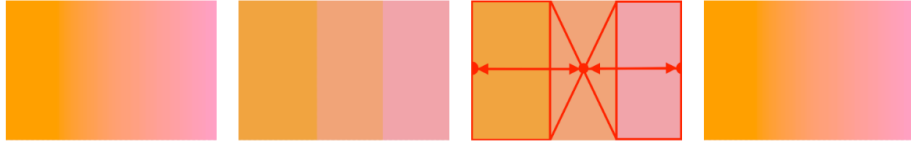


Fig. 2: Example of how mutation works for gradient path joining. From left to right: the original image; the image vectorized by a deterministic algorithm; the process of applying the gradient mutation; the vectorized image.

The edge of the shape area is taken as the gradient start points. If there are more than two parts, then the area's edge is taken only from the first part and the last one. For other parts, the center of mass is taken. This mutation creates the images with gradient coloring, which helps render the colors of the original image more accurately.

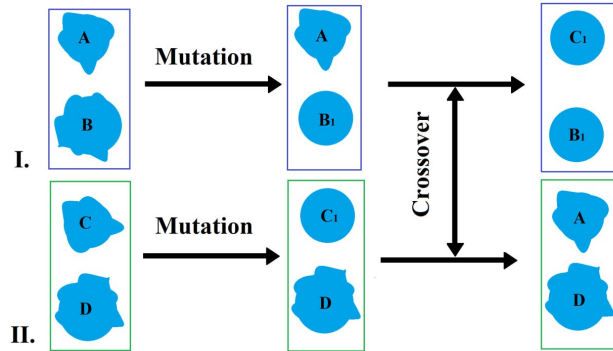


Fig. 3: Example of random path exchange crossover.

We use only one crossover for probabilistic path exchange. To achieve better quality, two individuals can exchange random paths with a certain probability. The example is presented in Figure 3.

Assume that two individuals I and II appeared after the mutation, let each individual has one good path and the other has bad path. Next, a crossover exchange of random paths is applied. For example, it chooses path A from individual I and path C_1 from II , then swaps them. As a result, two new individuals are created, and individual I is much better due to the crossover.

3.3 Selection function

Three selection function [16] were investigated. The selection function 1 is the pixel-by-pixel difference between the original bitmap image N and the current generated one M , formulated as follows:

$$F_1(N, M) = \sum_{i=0}^w \sum_{j=0}^h |N_{i,j} - M_{i,j}|, \quad (1)$$

where w — width of image, h — height of image.

The second selection function 2 is the same, but the difference between pixels is calculated exponentially.

$$F_2(N, M) = \sum_{i=0}^w \sum_{j=0}^h e^{|N_{i,j} - M_{i,j}| * \lambda + c}, \quad (2)$$

where $\lambda = \frac{50}{255}$, $c = 1$ to get a more sensitive difference color map.

In the third selection function 3, the difference between pixels in F_1 is squared.

$$F_3(N, M) = \sum_{i=0}^w \sum_{j=0}^h (|N_{i,j} - M_{i,j}| * \lambda)^2. \quad (3)$$

where $\lambda = \frac{1}{255}$ for normalization pixel colors.

This selection function is close in meaning to MSE [5]. As a part of the testing on different images, it was decided to focus on the third variant of selection function. This allows to account for stronger color differences better, while if the color is close to the original one, then the value function will be small.

In order to calculate the value of the selection function, a rasterizing of a vector image is required. We also studied and tested various Python libraries for rasterization (Aspose⁵, Wand⁶, CairoSvg⁷). CairoSvg is the best library for this task due to its speed and quality of rasterization.

⁵ <https://www.aspose.com/>

⁶ <https://pypi.org/project/Wand/>

⁷ <https://pypi.org/project/svgtrace/>

3.4 Color correction

To improve the algorithm operation, the color correction function is added. Often, the original colors of the deterministic algorithm may differ from the original ones. To solve this problem, a special color correction algorithm was created. It selects pixel colors for which the difference between them and the original ones is greater than a certain hyperparameter responsible for the sensitivity of the correction. Next, it selects all paths containing such colors and chooses the color for this path, which prevails in this section of the image. The implementation of the method can be seen in the algorithm’s details.

4 Comparisons and Experiments

As a part of the work, various experiments were carried out to identify the best parameters of the algorithm, as well as comparison with other existing solutions. Additional functions have been developed to simplify and improve the operation of the algorithm and the experimental setup has been described in detail.

4.1 Experimental setup

The main hyperparameters and thresholds are shown below. The number of individuals in one population is 30. The percentage of the elite is 10%. The number of iterations of the algorithm is 300. These parameters were derived empirically based on a large number of experiments.

All of the experiments were launched on CPU with 16 GB RAM and 10 cores. No GPUs have been used.

4.2 Comparison with machine learning algorithms

Firstly, we compare algorithms based on machine learning — LIVE and DiffVG — with our algorithm in terms of execution time, quality, and number of paths. Note that for LIVE $N = 16$, for DiffVG $N = 512$, where N is the predefined number of paths. This constant was chosen in such a way that the working time of the machine learning algorithms was comparable to the working time of our algorithm. Hyperparameters of these algorithms are set to default values. For DiffVG and LIVE N is necessary parameter, but our algorithm does not need it.

Table 1 shows that algorithms based on machine learning have a longer working time than our algorithm. For DiffVG and LIVE, the number of equal pixels in raster and vectorized images is significantly less, which indicates a low quality of vectorization. The number of paths for images vectorized by machine learning algorithms is equal to the initial constant, they have not to be compared by this parameter.

Table 1: Comparison of our algorithm with existing vectorization algorithms. The table shows metrics (vectorized image, working time, fitness value and number of paths) of algorithms: LIVE [15], DiffVG [14], Pixel2Svg, SvgTracer, and our algorithm.

Initial image	LIVE	DiffVG	Pixel2Svg	SvgTracer	Our Algo	Metrics
						Result
	51.3 m.	45.4 m.	3.9 s.	7.5 s.	24.5 m.	Time
	4760	4255	3278	1780	1547	Fitness
	35	512	1146	1790	985	Paths
						Result
	33.5 m.	38.4 m.	17.4 s.	1.9 s.	22.1 m.	Time
	4864	10720	4939	830	793	Fitness
	16	512	1789	2187	628	Paths
						Result
	2.7 m.	4.2 m.	1 s.	1.5 s.	2.6 m.	Time
	33	146	89	286	247	Fitness
	5	512	605	55	7	Paths
						Result
	72.4 m.	15.24 m.	1.1 m.	2.1 s.	5 m.	Time
	16290	15090	1626	1558	1550	Fitness
	16	512	6311	2947	1660	Paths
						Result
	77.3 m.	34.48 m.	10.3 s.	1.6 s.	29.2 m.	Time
	7503	15344	1501	1182	672	Fitness
	16	512	1522	798	438	Paths

4.3 Comparison with deterministic algorithms

Below we present a comparison of our algorithm with a deterministic algorithm based on various metrics. Pixel2Svg and SvgTracer were chosen as a deterministic algorithms, as they are powerful python libraries for image vectorization.

Table 1 clearly shows the advantage of our approach. The number of equal pixels between the original raster image and the vectorized one is higher due to gradient mutation. The proposed algorithm result contains fewer paths than others, due to the path deletion mutation, which means less weight of the vectorized image and interpretability. Our algorithm loses out in terms of execution time to deterministic algorithms, since at each iteration it is required to calculate the value of the selection function, which is a complex computational operation.

Table 2: Relative performance comparison of our algorithm with deterministic algorithms.

Size	Pixel2Svg	SvgTracer	Comparison criterion is relative to our algorithm
128x128	$-1569.8 \pm 67.7 \%$	$-500.7 \pm 38.9\%$	Number of paths
	$-69.6 \pm 4.2 \%$	$-16.9 \pm 4.1 \%$	Fitness
256x256	$-164.8 \pm 13.5\%$	$-103.6 \pm 10.1\%$	Number of paths
	$-27.9 \pm 2.5\%$	$-13.8 \pm 1.3\%$	Fitness
512x512	$-80.4 \pm 9.4\%$	$-67.1 \pm 8.9\%$	Number of paths
	$-18.8 \pm 1.7\%$	$-12.1 \pm 1\%$	Fitness
1024x1024	$-29.3 \pm 5.7\%$	$-24.2 \pm 4.7\%$	Number of paths
	$-9.6 \pm 0.9\%$	$-6.8 \pm 0.2\%$	Fitness

Our evaluation of the algorithm was conducted on image groups comprising 100 images each, with varying resolutions: 128x128, 256x256, 512x512, and 1024x1024 pixels. As demonstrated in Table 2, our algorithm exhibits a decrease in the computational paths required and an enhancement in the efficacy of the selection function when compared to the performance metrics of existing algorithms.

4.4 Ablation study

In this section, we discuss the importance of the proposed mutations and how they affect the final result [18]. Each mutation is probabilistic, that is why the test results for the same hyperparameters may differ.

The main effect of the needle mutation is changing the path shape by changing their coordinates. This is useful if the vectorized shape does not look very

Table 3: An impact example of the individual mutations on the ultimate outcome. From left to right: the initial image for vectorization; the image is vectorized by our algorithm; the image is vectorized by our algorithm without needle mutation; the image is vectorized by our algorithm without mutation for deleting paths and segments; the image is vectorized by our algorithm without mutation of gradient fusion.

Initial image	All mutations	W/o needle	W/o deletion	W/o gradient
				
Fitness	628	736	796	698
Number of paths	793	793	2187	793

similar to the expected one. Needle mutation was considered of two types. With a constant coefficient of change and uniformly decreasing. The first type proved to be better in the tests, and was later used in the research.

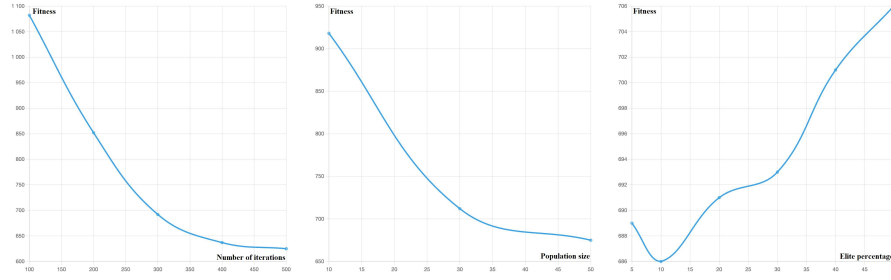
The removal of segments and paths is necessary to get rid of artifacts that occur during basic vectorization. It helps simplify the unnecessary complexity and the weight of the image, which is extremely convenient. However, there is an important addition: the deleted area should be less than or equal to the threshold responsible for the maximum square of the deleted area. This was done to avoid the problems associated with the complete removal of image important parts.

The last mutation, which gradiently connects the two or more paths, is also needed to simplify the structure of the images and for a clearer match to the originals due to the smooth transitions characteristic of bitmap images. The threshold between the color difference of paths can be tuned.

Thus, each of the mutations is responsible for a certain part of the improvement of the vectorized image. For example, Table 3 illustrates problems created by the absence of the proposed mutations. Their combination allows to achieve better vectorization results.

An important part of the work was to choose the right population size, the number of algorithm iterations, and the elite percentage. Table 4 shows experiment results with various population size and the iterations number. Based on this table, it can be concluded that a larger number of individuals in the population and many algorithm iterations can create a vectorized image with higher quality. It is worth noting that with an increase in these parameters, the quality ceases to improve (see Fig. 4a,4b), so we chose the number of individuals in one population as 30 and the number of the algorithm iterations as 300.

Figure 4c shows the influence of the elite percentage on the final vectorized image. The algorithm working time does not depend on the percentage of the



(a) Fitness dependence on the iterations number (b) Fitness dependence on the population size (c) Fitness dependence on the elite percentage

Fig. 4: Fitness dependencies on various factors.

Table 4: The dependence of the algorithm final result on the population size and the iterations number.

	Iterations number of the algorithm						Population size
	100	200	300	400	500		
Time	3.4 m.	6.7 m.	10.1 m.	13.4 m.	16.9 m.	10	
Fitness	1095	1004	918	870	834		
Paths	784	679	603	566	532		
Time	10.1 m.	20.3 m.	30.3 m.	40.5 m.	50.6 m.	30	
Fitness	1082	852	692	637	625		
Paths	781	604	473	426	413		
Time	16.9 m.	33.4 m.	50.7 m.	1.12 h.	1.4 h.	50	
Fitness	1078	860	687	627	616		
Paths	780	602	457	411	401		

elite, but the vectorized image quality and the paths number depends on it. Thus, the elite percentage equal to 10% is the best.

5 Conclusion

We have presented an evolutionary approach designed for the raster images vectorization using a variable number of curves. Unlike existing algorithms our solution is able to work with gradient colors. Additionally, it achieves superior quality results within an acceptable timeframe compared to other existing vectorization algorithms such as LIVE, DiffVG, Pixels2Svg, and SvgTracer. To evaluate the quality of vectorization, we employ a pixel-by-pixel Mean Squared Error (MSE)

loss calculation. This allows us to quantitatively measure the similarity between the vectorized image and the original bitmap image.

In order to further enhance the algorithm’s performance, we plan to incorporate new types of mutations and crossovers. These additions will address the current limitations, such as long operation time, and contribute to improving the algorithm accuracy. Another improvement involves increasing the algorithm’s speed by integrating alternative selection functions. By exploring different selection strategies, we aim to optimize the algorithm’s efficiency and reduce processing time.

Furthermore, we plan to directly operate with the contours of vectorized images. This approach will enable us to calculate the quality of vectorized images without the need for excessive rasterization. This advancement will simplify the evaluation process and increase the overall efficiency of the algorithm.

Acknowledgments

The research was supported by the ITMO University, project 623097 ”Development of libraries containing perspective machine learning methods”.

References

1. Back, T.: Evolutionary algorithms in theory and practice: evolution strategies, evolutionary programming, genetic algorithms. Oxford university press (1996)
2. Bellamy-Royds, A., Cagle, K.: SVG Colors, Patterns & Gradients: Painting Vector Graphics. ” O’Reilly Media, Inc.” (2015)
3. Bergen, S., Ross, B.J.: Automatic and interactive evolution of vector graphics images with genetic algorithms. *The Visual Computer* **28**(1), 35–45 (2012)
4. Bhandari, D., Murthy, C., Pal, S.K.: Genetic algorithm with elitist model and its convergence. *International journal of pattern recognition and artificial intelligence* **10**(06), 731–747 (1996)
5. Borovikov, V.: Statistica. the art of analyzing data on a computer: For professionals. st. petersburg: Piter; 2003
6. Chang, R.I., Su, C.Y.: Color gradient vectorization for svg compression of comic image. *Journal of Visual Communication and Image Representation* **33**, 235–246 (2015)
7. Dori, D., Liu, W.: Sparse pixel vectorization: An algorithm and its performance evaluation. *IEEE Transactions on pattern analysis and machine Intelligence* **21**(3), 202–215 (1999)
8. Dziuba, M., Jarsky, I., Efimova, V., Filchenkov, A.: Image vectorization: a review. arXiv preprint arXiv:2306.06441 (2023)
9. Frosini, P., Giorgi, D., Melzi, S., Rodolà, E.: Efficient image vectorisation using mesh colours
10. Gotshall, S., Rylander, B.: Optimal population size and the genetic algorithm. *Population* **100**(400), 900 (2002)
11. Jain, A., Xie, A., Abbeel, P.: Vectorfusion: Text-to-svg by abstracting pixel-based diffusion models. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. pp. 1911–1920 (2023)

12. Kazimipour, B., Li, X., Qin, A.K.: A review of population initialization techniques for evolutionary algorithms. In: 2014 IEEE congress on evolutionary computation (CEC). pp. 2585–2592. IEEE (2014)
13. Laine, S., Karras, T.: High-performance software rasterization on gpus. In: Proceedings of the ACM SIGGRAPH Symposium on High Performance Graphics. pp. 79–88 (2011)
14. Li, T.M., Lukáč, M., Gharbi, M., Ragan-Kelley, J.: Differentiable vector graphics rasterization for editing and learning. *ACM Transactions on Graphics (TOG)* **39**(6), 1–15 (2020)
15. Ma, X., Zhou, Y., Xu, X., Sun, B., Filev, V., Orlov, N., Fu, Y., Shi, H.: Towards layer-wise image vectorization. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 16314–16323 (2022)
16. Majig, M., Fukushima, M.: Adaptive fitness function for evolutionary algorithm and its applications. In: International Conference on Informatics Education and Research for Knowledge-Circulating Society (icks 2008). pp. 119–124. IEEE (2008)
17. Meißner, M., Bartz, D., Günther, R., Straßer, W.: Visibility driven rasterization. In: *Computer Graphics Forum*. vol. 20, pp. 283–293. Wiley Online Library (2001)
18. Meyes, R., Lu, M., de Puiseau, C.W., Meisen, T.: Ablation studies in artificial neural networks. *arXiv preprint arXiv:1901.08644* (2019)
19. Packard, K., Worth, C., Esfahbod, B.: Cairo
20. Quint, A.: Scalable vector graphics. *IEEE MultiMedia* **10**(3), 99–102 (2003)
21. Reddy, P., Gharbi, M., Lukac, M., Mitra, N.J.: Im2vec: Synthesizing vector graphics without vector supervision. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 7342–7351 (2021)
22. Spears, W.M., et al.: Adapting crossover in evolutionary algorithms. In: *Evolutionary programming*. pp. 367–384 (1995)
23. Tian, X., Günther, T.: A survey of smooth vector graphics: Recent advances in representation, creation, rasterization and image vectorization. *IEEE Transactions on Visualization and Computer Graphics* (2022)
24. Wang, Y., Lian, Z.: Deepvecfont: synthesizing high-quality vector fonts via dual-modality learning. *ACM Transactions on Graphics (TOG)* **40**(6), 1–15 (2021)
25. Wenyin, L., Dori, D.: From raster to vectors: Extracting visual information from line drawings. *Pattern Analysis & Applications* **2**, 10–21 (1999)
26. Young, S.R., Rose, D.C., Karnowski, T.P., Lim, S.H., Patton, R.M.: Optimizing deep learning hyper-parameters through an evolutionary algorithm. In: Proceedings of the workshop on machine learning in high-performance computing environments. pp. 1–5 (2015)
27. Zhou, H., Zheng, J., Seah, H.S.: Converting bitmap images into scalable vector graphics. In: Proceedings of the Korean Society of Broadcast Engineers Conference. pp. 435–440. The Korean Institute of Broadcast and Media Engineers (2009)
28. Zhou, Y., Li, X., Gao, L.: A differential evolution algorithm with intersect mutation operator. *Applied soft computing* **13**(1), 390–401 (2013)